CSI v10 — Extending the Application with Mongoose

Study notes from the Infor official training workbook (214 pages)

Why this guide matters

This is the primary **developer course** — the step beyond personalizing forms. It teaches how to extend CSI at the data layer: creating new database tables, IDOs, and forms from scratch; extending existing IDOs and forms; building portal-facing screens; and working safely in a multi-developer environment. If you need to add a field that doesn't exist anywhere in the application, or build a brand-new business process, this is the knowledge you need.

Prerequisites: UI Navigation and Creating Form Personalizations. Those courses cover the client tier. This course works primarily at the IDO (middle) tier.

---

1. The Mongoose Three-Tier Architecture

All CSI development happens inside the Mongoose framework's three tiers. Knowing which tier owns which concern is essential before writing a line of customization code.

Tier	What lives here / what happens
1 — Client (WinStudio Smart Client or Web Client)	Forms, components, form-level event handlers. WinStudio metadata (form definitions) is stored in a Forms database and loaded at runtime. Smart Client and Web Client render the same metadata differently.
2 — Middle Tier (IDO Runtime Service)	IDO business objects, property classes, IDO methods, custom C# extension class DLLs. The IDORuntimeHost.exe process runs here. Every client request (load, save, invoke) is handled by this tier before touching the database.
3 — Database (SQL Server)	Application data tables, stored procedures, views. Also stores: IDO metadata, form metadata, Application Event System tables, report definitions. Custom stored procedures and triggers live here.

Important mental model: A form Save calls UpdateCollection on the IDO Runtime (tier 2), which executes SQL on tier 3. Custom C# extension class logic runs on tier 2 — between the client request and the SQL execution.

---

2. Development Permission Levels — All Five

Editing permission is set per-user on the Users form. Check yours: **Help > About <servername>**.

Level	What they can do
Vendor Developer (value 4, set manually — hidden from dropdown)	Bypasses all customizations; sees pure vendor defaults. Use this to diagnose whether a bug is in the base product or a customization. Never make real development changes here.

Level	What they can do
Site Developer	Full development power: create/delete forms and IDOs at any scope (Vendor/Site/Group/User), impersonate any user/group to view their version, manage all global objects. This is your day-to-day developer level.
Full User	Can fully customize forms at their own User scope only. Cannot create or delete forms, and cannot affect other users.
Basic User	Simple user-scope changes only: hide fields, set read-only, change colors and fonts, rearrange components. Cannot create or delete forms.
None	Cannot enter Design Mode at all. The design mode icon is disabled. Default for standard end users.

Entering Design Mode: Clear the filter-in-place first, then press **Ctrl+E** or click the Design Mode icon. A Site Developer will see a scope dialog — always verify you have the correct scope (Vendor / Site Default / Group / User) before saving anything.

---

3. IDOs (Intelligent Data Objects) — The Core Building Block

An IDO is a named business object on the middle tier that encapsulates a set of database columns, exposes them as named properties to the client, and defines the methods (business logic) for interacting with those columns. Every WinStudio form collection is bound to exactly one IDO.

IDO anatomy

Element	What it is
Table References	One or more SQL tables or views the IDO reads and writes. The IDO maps columns to named properties.
Property Definitions	Named columns/calculated values exposed to the client. Key attributes per property: Name, Data type, Length, Required/Optional, Key flag (marks primary key columns), Read-only, Calc (derived/not stored).
Property Classes	Reusable attribute templates. If 15 IDOs all have a 'Status' field with the same type and behavior, they all inherit from one Property Class — change the class and all IDOs update automatically.
IDO Methods	Custom stored procedures or C# methods attached to the IDO. Types: Fetch (returns rows), Non-fetch (writes without returning rows), GetProperty, SetProperty. Invoked via the Invoke() built-in method.
IDO Extension Class	Optional C# DLL that overrides LoadCollection, UpdateCollection, or Invoke at the middle tier. Required for complex cross-IDO validation, external integrations, or logic that cannot live in a stored procedure.

Standard built-in methods — every IDO has these four

– **LoadCollection** — retrieves rows matching a filter; returns them to the client.

– **UpdateCollection** — handles insert, update, and delete for a batch of rows.

– **GetPropertyInfo** — returns property metadata (types, lengths, required) to the form.

– **Invoke** — calls a named custom method, passing and returning parameters.

Inheritance chain for property attributes (broadest → most specific)

IDO Property Class → IDO Property → Property Class Extension → Component Class → Component (instance on form)

The most specific level always wins. A component-level override beats everything above it.

---

4. Creating New Functionality — The New Data Maintenance Wizard

When you need a brand-new entity (new SQL table + IDO + form), the New Data Maintenance Wizard creates all three in a guided sequence.

What the wizard creates	Details
SQL Table	A new table with your defined columns plus the standard system columns (RowPointer, CreateDate, CreateUser, ModifyDate, ModifyUser, InWorkflow).
IDO Definition	An IDO bound to the new table, with properties corresponding to each column.
Multiview Form	A two-pane WinStudio form (grid list on top, detail below) bound to the new IDO. Immediately usable for CRUD operations.

Post-wizard refinement: Set the primary key; mark required fields; add dropdown list sources (list sources); add validation logic in the IDO or extension class; refine form layout and tab order; add to the Explorer navigation.

Access path: In Design Mode → System > Form > Definition > New (choose 'Build New Data Maintenance Form').

---

5. Extending Existing IDOs — Adding Custom Data

The most frequent developer task: adding custom columns to existing application tables that Infor owns. Two supported approaches:

Approach A — User Extended Tables (UETs) [preferred, upgrade-safe]

UETs add physical columns to existing application tables without modifying Infor's base IDO definition. Infor upgrades do not remove UET columns.

Step	Form to use / action
1. Define a UET Class	UET Classes form — give the set of new fields a name.
2. Define UET Fields	UET User Fields form — each field: name, data type (e.g. nvarchar(50)), length.
3. Link fields to class	UET Class/Field Relationships form.

Step	Form to use / action
4. Link class to table	UET Table/Class Relationships form — specify the real table (e.g. co_mst).
5. Apply schema	UET Impact Schema form → click Process. Physically adds columns to SQL Server.
6. Refresh metadata	Ctrl+U (with 'Unload IDO Metadata With Forms' on) or restart IDO Runtime Service.
7. If using replication	Replication Management form → Regenerate Replication Triggers.

Approach B — IDO Property Extension at Site scope

Add properties directly to an existing vendor IDO at site scope. The vendor base IDO is untouched; your extension properties are merged at runtime. Use when UETs are insufficient (e.g., adding a calculated/joined property).

---

6. Extending Existing Forms

A Site Developer can add components, event handlers, and tab pages to any vendor form at Site scope — without touching the base vendor definition.

Task	How to do it
Add a new field/component	Drag from Toolbox → draw on form. Bind to an IDO property: Component sheet > Data Source > Binding. The IDO property must exist first (add it to the IDO if needed).
Add an event handler	Edit > Event Handlers → New. Set Event Name, Sequence, Response Type (e.g. Method Call, Set Values, Generate Application Event, Goto Form).
Add a NoteBook tab	Add a NoteBook component; add NotebookTab children inside; assign captions; set tab order (select notebook before tabs inside).
Override a base event handler	Set 'From Base Form' = False on the inherited handler. Warning: this re-associates ALL handlers for that event — review the full list. Only override what you need to change.
Extended vs. copied form	Extension: inherits future base-form upgrades (preferred). Copy: fully independent — you own all maintenance. Prefer extension unless you need full independence.

---

7. Portal Forms — External User Access

The Customer Portal and Vendor Portal let external users interact with the application via a browser, without WinStudio. The course uses an **Expense Report submission form** as the worked example.

Concept	Detail
Enable a form for portal	Set form property 'Display In Portal' = True.
Control visible fields	Mark fields Hidden = True in the portal version to restrict what external users see.

Concept	Detail
Enforce data security	Use IDO-level permanent filter expressions so each portal user only sees their own data. UI hiding alone is never sufficient security.
Workflow integration	Typical pattern: external user submits via portal → Application Event System fires → internal manager approves in WinStudio → record commits to database.

---

8. Team Development — Check-In / Check-Out and FormSync

In a team, multiple Site Developers may need to work on the same form set. The check-out mechanism prevents simultaneous overwrites.

Action	What it does
Check Out	Locks the form definition for your exclusive edit. Others see 'Checked Out By [you]' and cannot save.
Check In	Releases the lock and publishes your changes to all users.
Discard Checkout	Abandons your in-progress changes and releases the lock without saving.
Force Check In (Vendor Dev only)	Releases another user's lock in an emergency.

FormSync — moving customizations between environments

FormSync exports/imports form definitions and global objects (component classes, validators, strings, form-level event handlers) as XML files. **FormSync does NOT cover:** IDO metadata, SQL tables/stored procedures, Application Event System event/handler metadata, report definitions — those require separate deployment procedures.

FormSync path: Master Explorer > Modules > System > Utilities > FormSync.

Quick Reference: Essential Design-Mode Paths

Task	Path / shortcut
Enter / exit Design Mode	Ctrl+E (after clearing filter-in-place), or toolbar button
New form + IDO + table wizard	System > Form > Definition > New
Copy a form	System > Form > Definition > Copy
Check out / check in	System > Form > Definition > Check Out / Check In
UET setup (all steps)	Master Explorer > Modules > System > User Extended Tables
UET Impact Schema	Master Explorer > Modules > System > UET Impact Schema
Discard IDO metadata cache	Ctrl+U (requires 'Unload IDO Metadata With Forms' in User Preferences > Runtime Behaviors)

Task	Path / shortcut
Restart IDO Runtime Service	Windows Services console on utility server > Infor Framework IDO Runtime Services
FormSync	Master Explorer > Modules > System > Utilities > FormSync
Check current permission level	Help > About